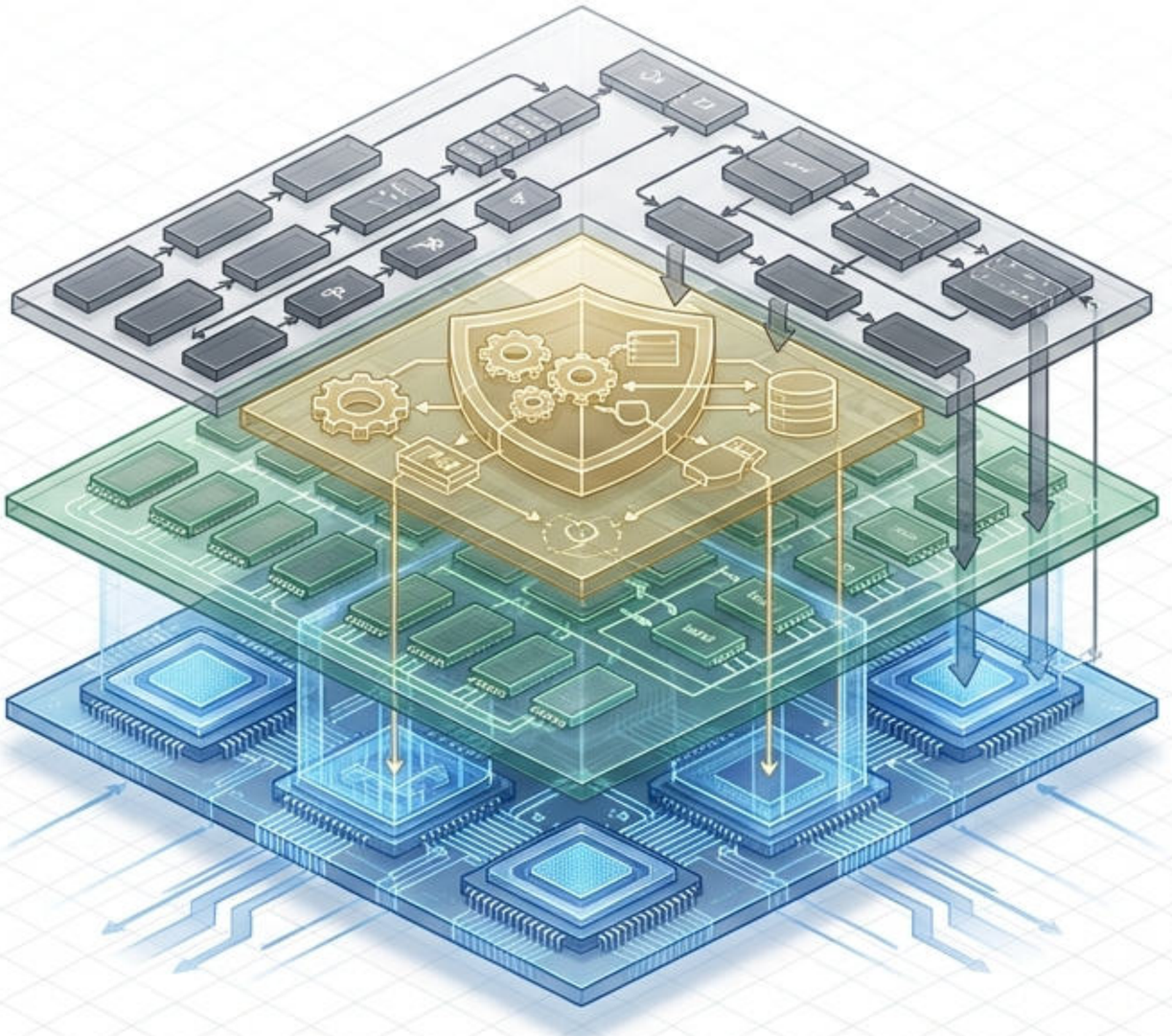
# מערכות מרובות מעבדים: ארכיטקטורה, סנכרון ותזמון

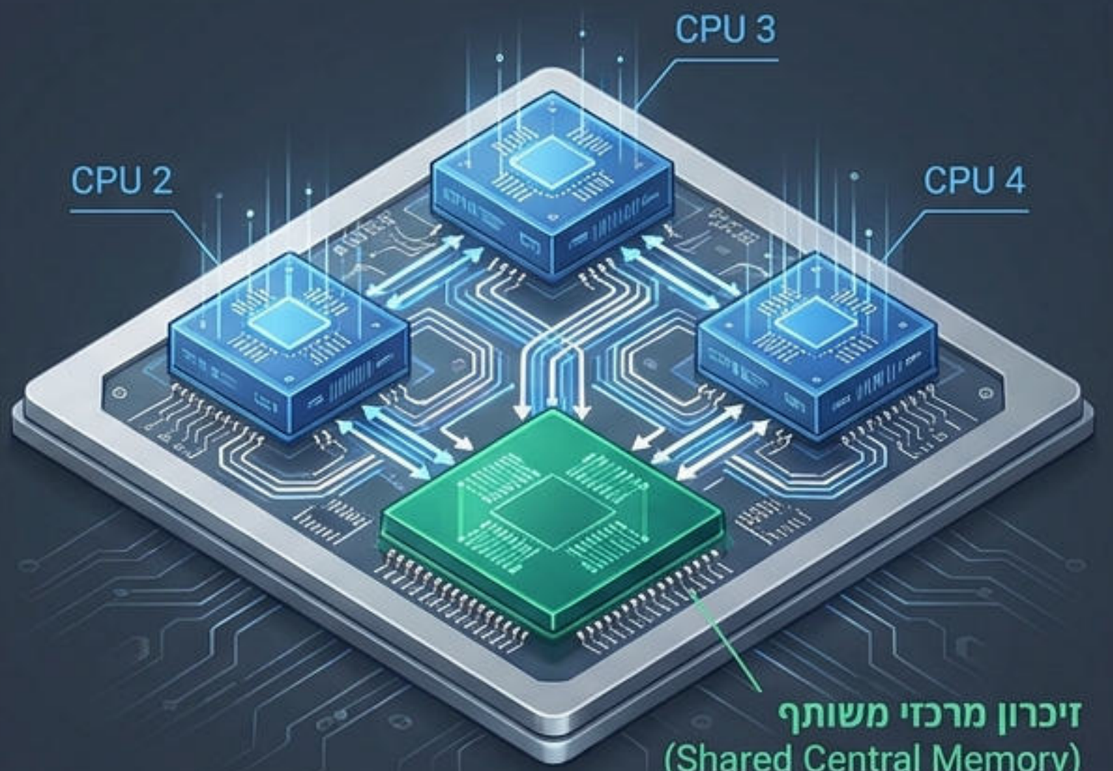
המדריך המקיף לניהול חומרה ותהליכים בסביבה מרובת ליבות (פרק 8)



# מגבלת הפיזיקה והמעבר לריבוי מעבדים




**האתגר:** הגענו למגבלות פיזיקליות של מהירות השעון במעבדים בודדים. הפתרון הוא מעבר למקביליות מבוססת חומרה.

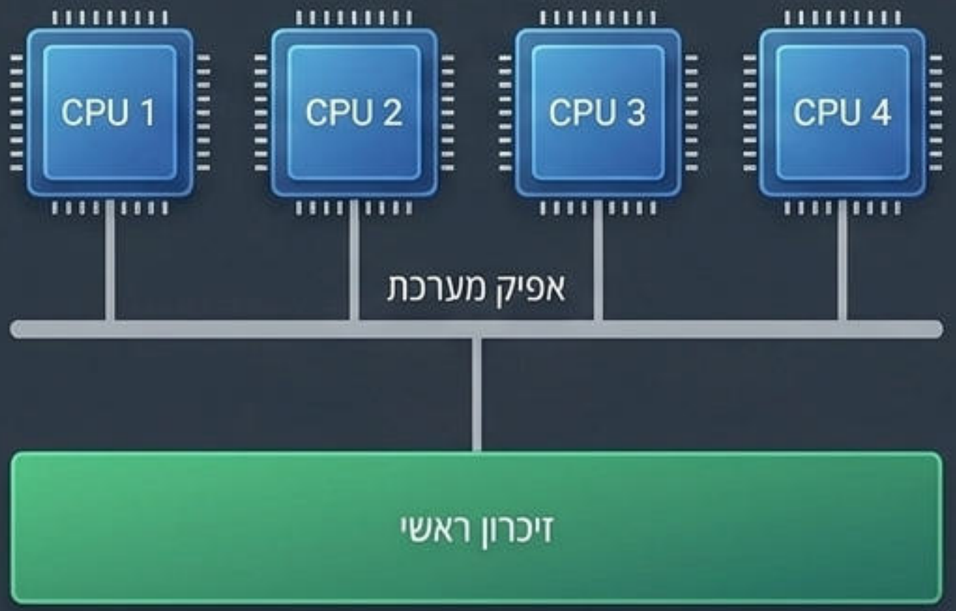



**הפתרון:** **מערכת מרובת מעבדים** (Multiprocessor) - מעבדים עצמאיים העובדים יחד להגדלת התפוקה תוך שיתוף הזיכרון.

**מושג מפתח:** **ליבות** (Multi-core) - מעבדים עצמאיים הבנויים על אותו שבב סיליקון (Die) בודד.

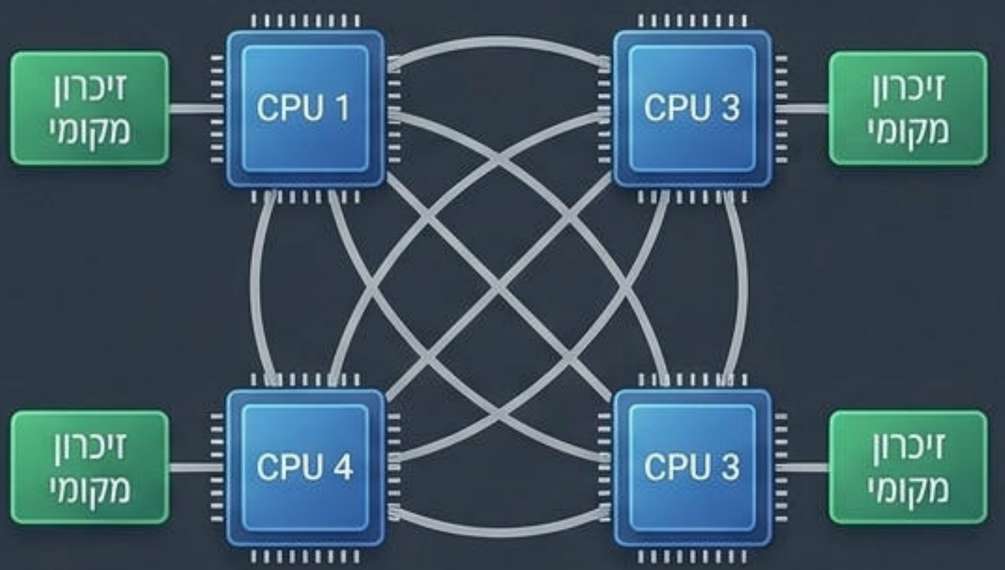
# ארכיטקטורות גישה לזיכרון: UMA מול NUMA

## **UMA** (Uniform Memory Access)



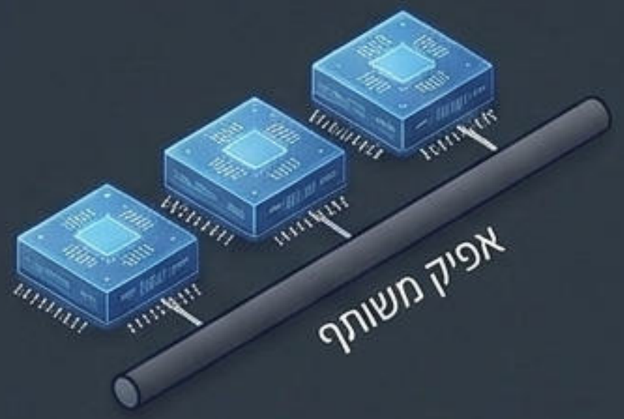

- **מבנה:** כל המעבדים מחוברים דרך אפיק מערכת משותף יחיד.
- **זמן גישה:** אחיד וזהה לכל כתובת בזיכרון הראשי.
- **יתרון וחסרון:** פשוט לתכנון וזול, אך האפיק הופך לצוואר בקבוק (עומס) ככל שמוסיפים ליבות.

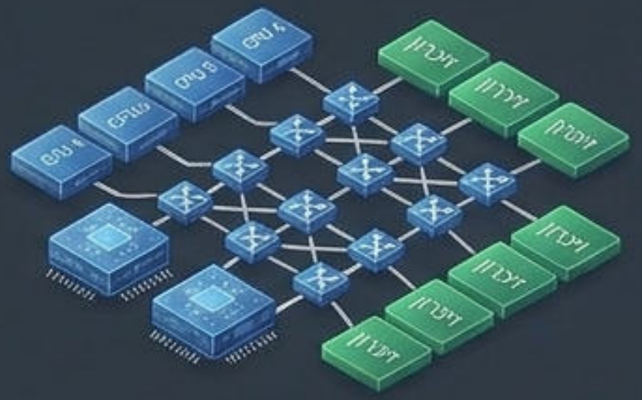
## **NUMA** (Non-Uniform Memory Access)

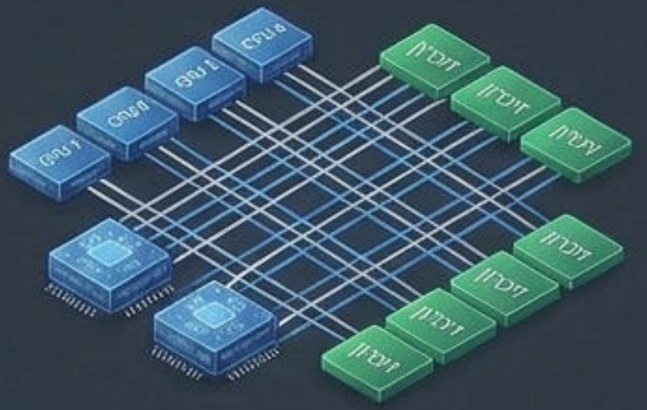



- **מבנה:** לכל מעבד זיכרון מקומי מהיר; כולם מקושרים ברשת חיבורים
- **זמן גישה:** לא אחיד. גישה מקומית מהירה מאוד, מרוחקת איטית יותר.
- **יתרון וחסרון:** פתרון מעולה וסקלאבילי למערכות מרובות מעבדים למניעת פקקי תנועה מרכזיים.

# טופולוגיות חומרה: ספקטרום החיבור הפיזי





פשוט וזול ← מורכב ויקר (Scalability)

**1. אפיק משותף (Bus)**
ערוץ יחיד. נוצר עומס קריטי המגביל כושר גדילה.

**2. רשת מיתוג (Multistage)**
מתגים קטנים המנתבים בשלבים. פשרה חכמה למערכות ענק.

**3. מתג צולב (Crossbar Switch)**
חיבור ישיר לכל רכיב ללא המתנה. ביצועים מצוינים אך עלות ריבועית ומסורבלת.

**שאלת עומס:** מה קורה כשכל הליבות ניגשות לאותה כתובת יחד (Memory Contention)?

החומרה מבצעת סריאליזציה (Serialization). האפיק מסדר את הבקשות לטיפול טורי (אחת אחרי השנייה) כדי למנוע התנגשויות.

# אשליה חומרתית: ריבוי תהליכונים (SMT / Hyper-threading)



יחידת חישוב פיזית אחת (ALU)

אוגרים ומונה תוכנית משוכפלים (מרחב מצב)

**העיקרון הלוגי:**

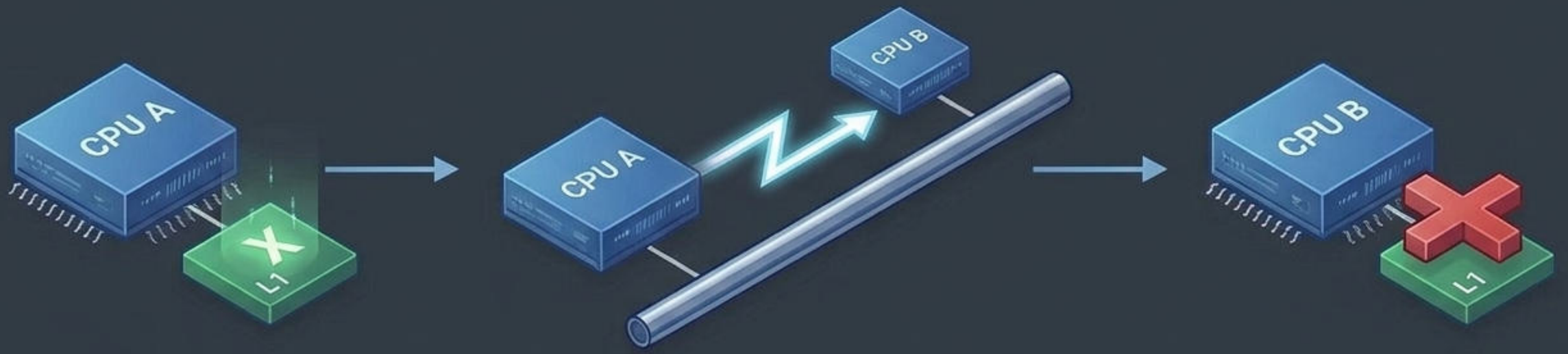
מערכת ההפעלה 'רואה' 2 מעבדים לוגיים שונים ויכולה לשלוח אליהם תהליכונים.

בפועל, החומרה מזפזופת ביניהם באפס זמן כאשר אחד מהם ממתין לנתונים מהזיכרון.

**⚠ אזהרת תזמון:**

שיבוץ של שני תהליכים כבדים (CPU-Bound) על אותה ליבה פיזית יוביל לכך שהם יחנקו זה את זה בתחרות על ה-ALU המשותף.

# אתגר העקביות: פרוטוקול מטמון (Cache Coherence)




**1.** מעבד A מעדכן משתנה X במטמון הפרטי שלו.

**2.** אות ביטול: מעבד A משדר הודעת Invalidate על האפיק.

**3.** פסילה: מעבד B פוסל את העותק הישן של X מהמטמון שלו.

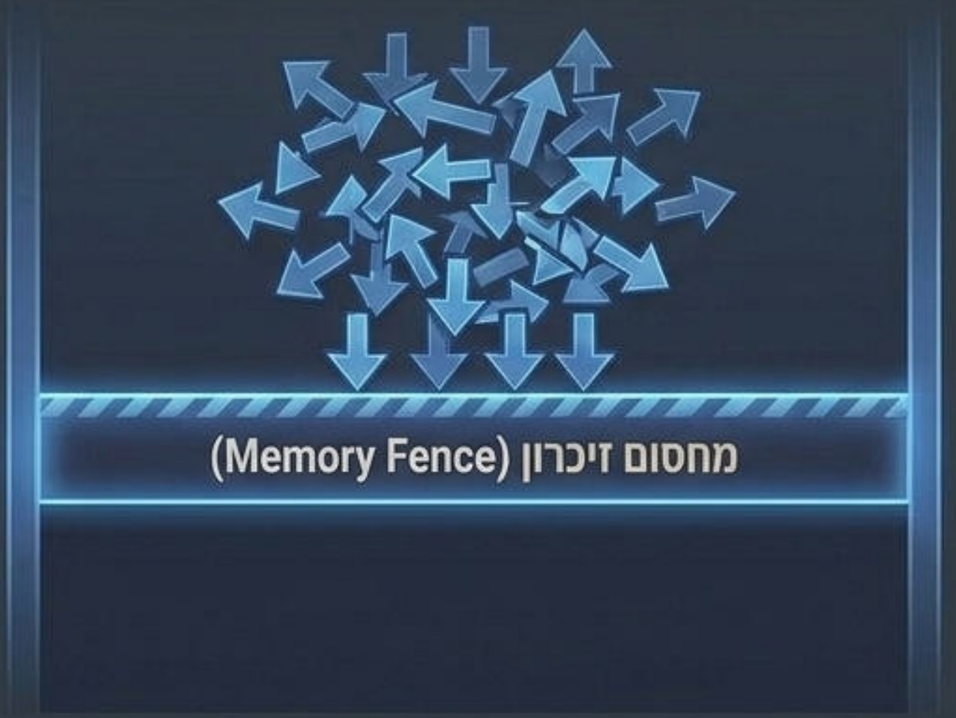
### שיטת ההאזנה (Snooping / MESI)

כל מעבד משדר באפיק. כולם 'מאזינים' ופוסלים מטמון מטמון בעת שינוי.
אידיאלי לארכיטקטורת UMA המבוססת אפיק משותף.

### שיטת הספריה (Directory)

רכיב מרכזי מנהל רישום של מי מחזיק איזה נתון.
רק הליבות הרלוונטיות מקבלות הודעת ביטול.
אידיאלי ל-NUMA למניעת עומס ברשת הפנימית.

# ביצוע מחוץ לסדר ומחסומי זיכרון (Memory Consistency)




**הבעיה:**

מעבדים מודרניים מבצעים פקודות מחוץ לסדר הכתיבה כדי לייעל את הביצועים, מה ששובר לחלוטין קוד מקבילי.

**הפתרון (Barriers):**

מחסום זיכרון מורה למעבד לעצור את ההתקדמות עד שכל פקודות הזיכרון הקודמות הופצו וסונכרנו לכל הליבות.

**אחריות המפתח:**

החומרה ומערכת ההפעלה לא ממקבלות קוד סדרתי אוטומטית. חובה על המתכנת לנהל את התהליכונים ולהשתמש במחסומים ומנעולים כראוי.

# מי שולט במערכת? 3 ארכיטקטורות של מערכות הפעלה

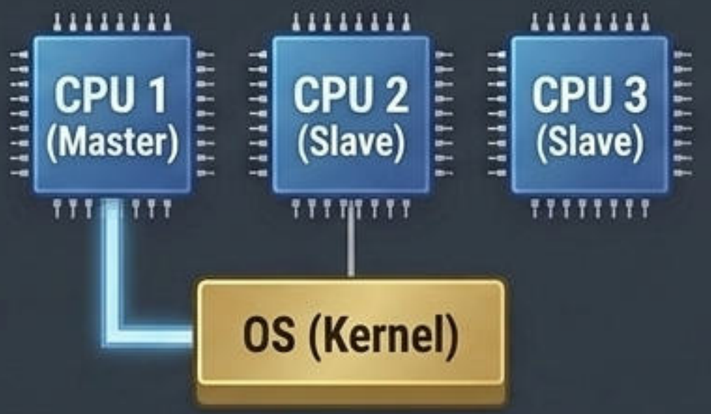
## 1. מערכת הפעלה לכל מעבד




הזיכרון מחולק סטטית למחיצות, עותק OS עצמאי לכל מעבד.

חסרון: חוסר גמישות קיצוני. מעבד אחד יכול לקרוס מעומס בעוד אחר בטל לחלוטין.

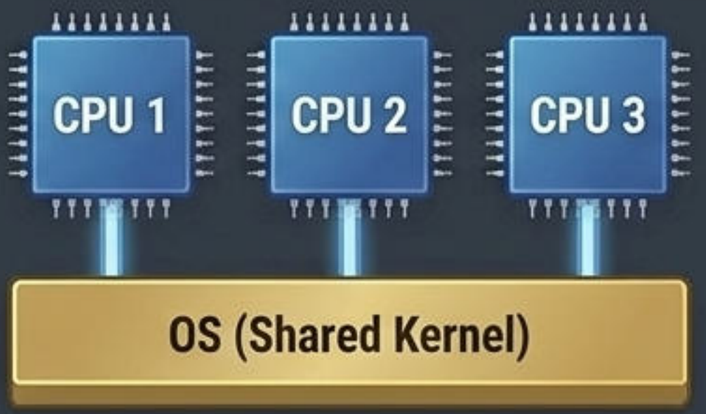
## 2. מוביל-נגרר (Master-Slave)




עותק יחיד. רק מעבד אחד (Master) מורשה להריץ קוד קרנל וקריאות מערכת.

חסרון: צוואר בקבוק. כשיש הרבה פניות למערכת, המוביל קורס מ-100% עומס והנגררים ממתינים.

## 3. ריבוי מעבדים סימטרי (SMP)




עותק יחיד, כל המעבדים שווי זכויות. כולם יכולים להריץ קוד קרנל בו-זמנית.

יתרון: מונע צווארי בקבוק ומהווה את הסטנדרט המודרני, אך דורש סנכרון והגנה קפדנית על נתונים.

# אשליית הסנכרון: למה לא פשוט לבטל פסיקות?



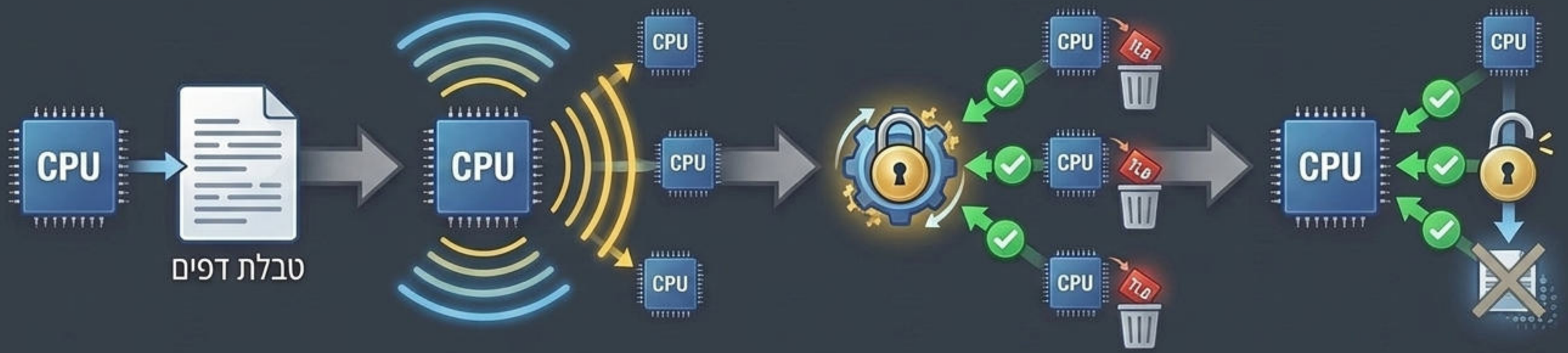
**חסימת פסיקות (Disabling Interrupts)**
**מקומית**. מעבד A מבטל פסיקות כדי למנוע החלפת הקשר (Context Switch).

**מעבד B ממשיך לרוץ!**
הוא אינו מושפע מחסימת הפסיקות של A ודורס את המידע בקטע הקריטי.




**המסקנה:** ב-SMP, ביטול פסיקות אינו מספק מניעה הדדית (Mutual Exclusion).
**נדרשים מנעולי חומרה אטומיים אמיתיים** למניעת התנגשות בזיכרון.

# צוואר הבקבוק המרכזי: פסיקה בין-מעבדית (TLB Shootdown)




1. מערכת ההפעלה מעדכנת **טבלת דפים** (מוציאה דף לדיסק).

2. **שידור אזהרה**: נשלחת פסיקה בין-מעבדית (IPI) לכל שאר הליבות במערכת.

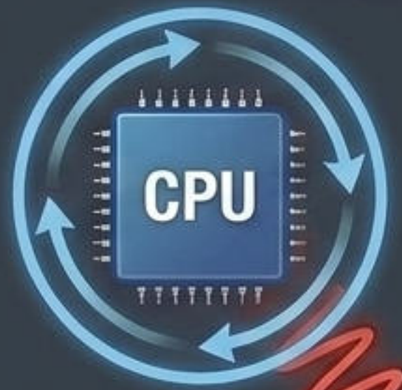
3. **השבתה והמתנה פעילה**: מעבד 1 נעל (Spinlock) וחייב לחכות ששאר הליבות ימחקו את ה-TLB שלהן ויאשרו.

4. **שחרור**: רק לאחר קבלת אישורים מכולם (ACK), הדף הפיזי משוחרר מהזיכרון הראשי.

**הערת ביצועים:** זהו צוואר בקבוק משמעותי במערכות מקביליות, בו ליבה אחת משתקת ליבות אחרות לסנכרון זיכרון וירטואלי.

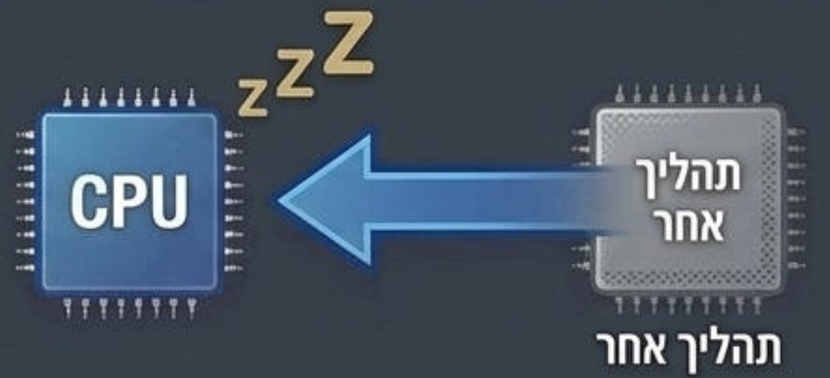
# בניית מנעולים (Locks): הכרעה אדריכלית ב-SMP

## Spinlock (המתנה פעילה)




- **אופן הפעולה:** המעבד דוגם את המנעול בלולאה אינסופית פעילה.
- **יתרון:** תגובה מיידית עם שחרור המנעול, ללא תקורת הרדמה.
- **חסרון:** חונק את אפיק הזיכרון בתעבורה, מבזבז כוח עיבוד על סרק.
- **שימוש אידיאלי:** קטעים קריטיים קצרים מאוד בחומרה.

## Mutex / Context Switch (הרדמה)




- **אופן הפעולה:** המעבד מורדם, והמתזמן בוחר תהליך אחר להרצה.
- **יתרון:** המעבד מתפנה לעבודה מועילה אחרת ואינו מעמיס על האפיק.
- **חסרון:** פעולת החלפת ההקשר היא איטית ויקרה במשאבים (תקורה).
- **שימוש אידיאלי:** קטעים קריטיים ארוכים (כגון קריאות I/O).

**הבסיס הפיזיקלי:** המנעולים מבוססים על פקודות חומרה כמו SWP המבצעות קריאה וכתיבה לזיכרון כפעולה אטומית שאינה ניתנת לחלוקה. שתי ליבות לעולם לא יקראו '0' במקביל.

# תזמון חכם: חשיבותה של זיקת מטמון (Cache Affinity)

**תרחיש א':**
חזרה לאותה ליבה




המתזמן מחזיר את התהליך לרוץ על אותה ליבה. התוצאה:
'מטמון חם' - הנתונים כבר קיימים בזיכרון המקומי והביצועים מקסימליים ומידיים.

---

**תרחיש ב':**
העברה לליבה חדשה



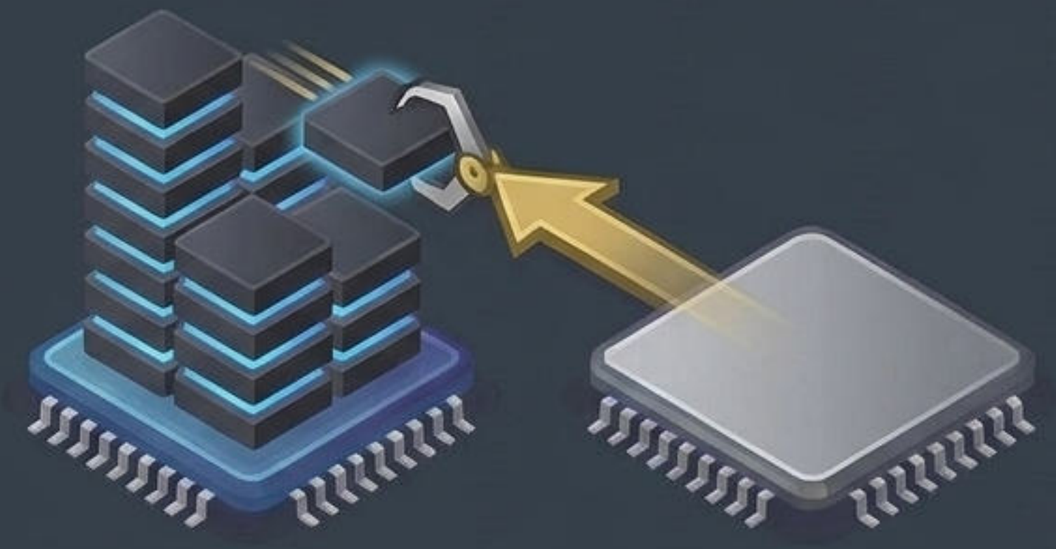

המתזמן מעביר את התהליך לליבה אחרת.
**התוצאה:** 'מטמון קר' - הליבה החדשה נאלצת להמתין לייבוא הנתונים מחדש מהזיכרון הראשי, דבר הפוגע משמעותית בביצועים.

**מסקנת התזמון:** מתזמן חכם שואף לשמור על Cache Affinity ולמנוע נדידת תהליכים היכן שאפשר.

# איזון עומסים אקטיבי (Load Balancing)

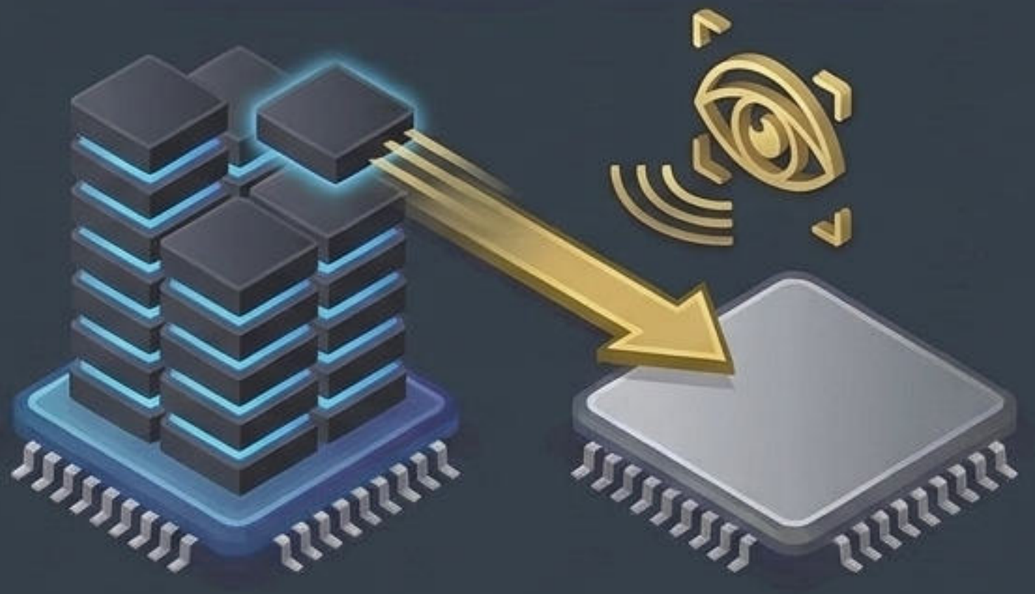
זיזיקת מטמון מושכת תהליך להישאר במקומו, אך התוצאה עלולה להיות מעבד עמוס מול מעבד בטל.
**איזון עומסים** פותר זאת בשתי גישות מנוגדות:

## משיכה (Pull Migration)



המעבד הפנוי יוזם בעצמו משיכה ('גניבת עבודה') ושולף משימה מתור ההמתנה של מעבד עמוס.

## דחיפה (Push Migration)



תהליך רקע של מערכת ההפעלה בודק עומסים, ודוחף בצורה אקטיבית משימות ממעבד עמוס אל מעבד פנוי.

# סנכרון בזמן ובמרחב: תזמון כנופיות (Gang Scheduling)

## שיתוף זמן קלאסי - חוסר תיאום גורר בזבוז משאבים




תהליך B מתבזבז בהמתנה פעילה כי תהליך A רדום ולא יכול לשחרר את המנעול.

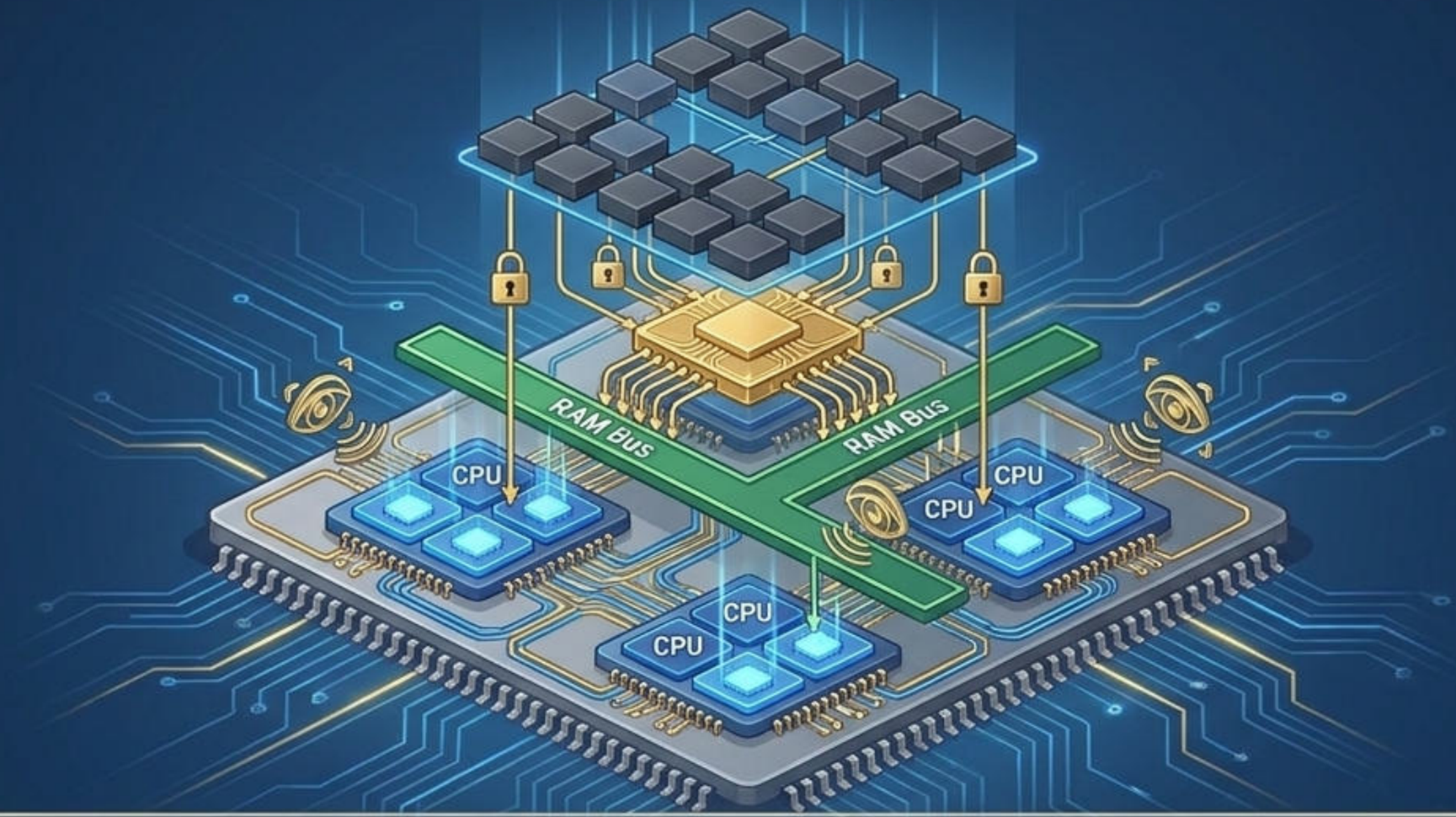
## תזמון כנופיות - התיאום המושלם




המתזמן מריץ את כל התהליכנים הקשורים יחד באותו זמן על ליבות שונות. עבודה בסנכרון מלא ללא המתנות סרק.

**מושג מקביל: שיתוף מרחב** (Space Sharing) - הקפאת קבוצת ליבות ומסירתן בלעדית לתהליך אחד כבד ללא הפרעות או החלפות הקשר כלל.

# ארכיטקטורת ריבוי מעבדים: תמונת המערכת המלאה




מערכות מרובות מעבדים אינן רק 'הכפלת חומרה', אלא מערכת אקולוגית מורכבת הדורשת תכנון מסונכרן. החל מטופולוגיית הזיכרון (NUMA/UMA), דרך פרוטוקולי עקביות מטמון ומחסומי זיכרון, ניהול שוויוני (SMP), וכלה בתזמון מרחבי המכיר את מגבלות הפיזיקה. האחריות הסופית לניצול הכוח המקבילי בצורה בטוחה ויעילה נחה תמיד בידיו של מפתח התוכנה.